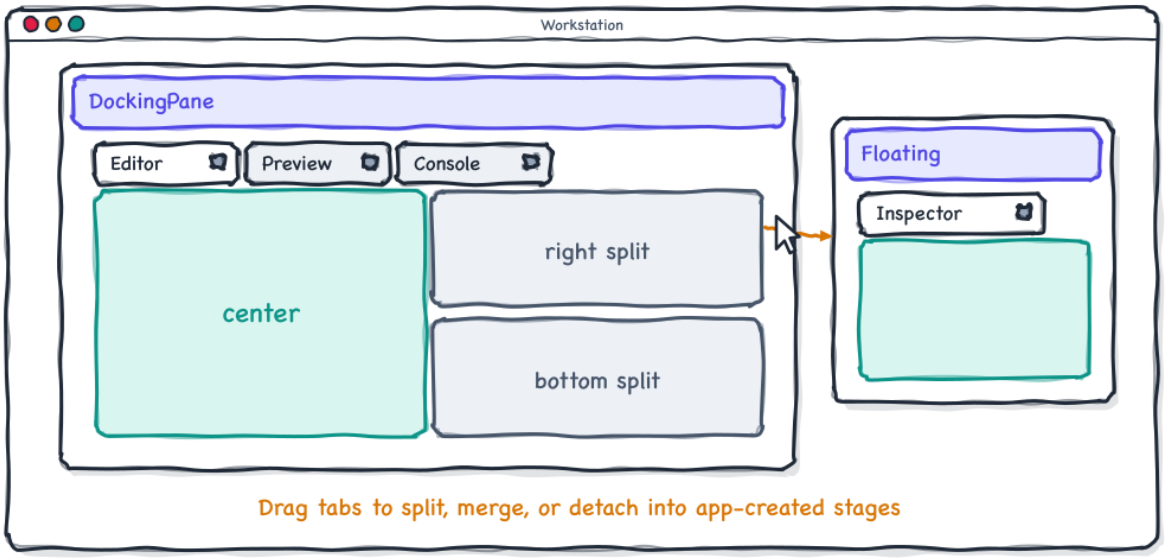
DockingPane

Developer Manual

GemsFX • com.dlsc.gemsfx • JavaFX Custom Controls



Dockables can be grouped as tabs, split into regions, and moved into floating stages.

DockingPane provides IDE-style docking with draggable tabs, split layouts, optional floating windows, and complete context snapshots through DockingContext. This manual covers structure, interaction, stage hosting, persistence hooks, styling, accessibility, and a practical application recipe.

Table of Contents

1. Introduction 2

1.1 Key features 2

1.2 Maven dependency 2

2. Getting started 2

3. Anatomy 3

4. Control API 4

4.1 DockingContext 4

4.2 DockingPane 4

5. Tab and split interactions 4

6. Floating stages 5

7. Persistence 5

8. Styling 6

9. Accessibility and localization 7

10. Basic app recipe 8

11. See also 8

1. Introduction

DockingPane is a JavaFX `Control` that renders dockables as custom title tabs (not a JavaFX `TabPane`). A `DockingContext` owns one or more panes, resolves dockables by ID, coordinates cross-pane movement, and creates floating windows through an application-supplied `DockingStageFactory`.

1.1 Key features

- Custom tab strip with title, optional graphic, and close button per dockable.
- Split layouts with horizontal/vertical panes and divider positions in [0, 1].
- Drag-and-drop within one pane and across panes in the same context.
- Detaching tabs into app-created floating stages.
- Context-wide snapshot and restore via `getLayout()` and `restoreLayout()`.
- Close policy hook via `setCanCloseDockable(...)`.

1.2 Maven dependency

```
<dependency>
    <groupId>com.dlsc.gemsfx</groupId>
    <artifactId>gemsfx</artifactId>
    <version>4.7.0</version>
</dependency>
```

DockingPane lives in module `com.dlsc.gemsfx`, package `com.dlsc.gemsfx`.

2. Getting started

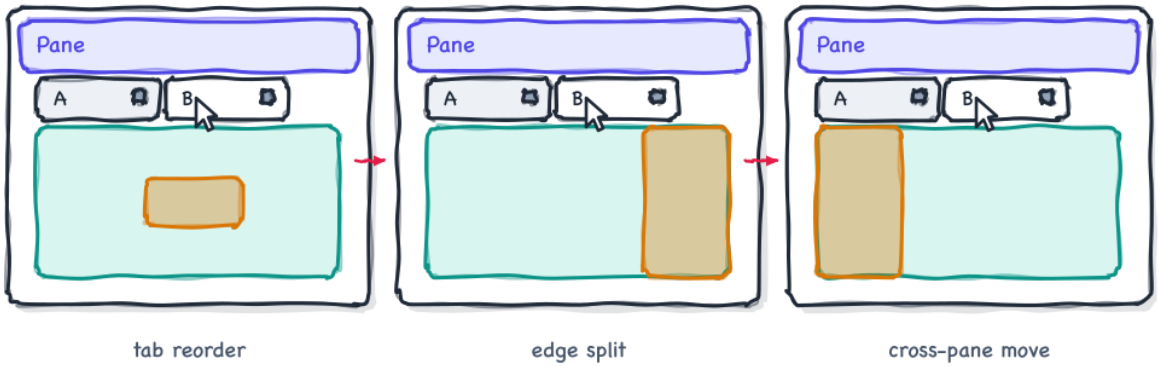
A docking setup starts with two factories: one that creates dockables by ID and one that returns a stage hosting each floating pane.

```
DockableFactory dockableFactory = id -> {
    if ("editor".equals(id)) {
        return Optional.of(new Dockable("editor", "Editor", new TextArea()));
    }
    if ("inspector".equals(id)) {
        return Optional.of(new Dockable("inspector", "Inspector", new ListView<>()));
    }
    return Optional.empty();
};

DockingStageFactory stageFactory = pane -> {
    Stage stage = new Stage();
    stage.setTitle("Floating Dock");
    stage.setScene(new Scene(new BorderPane(pane), 420, 280));
    stage.show();
    return stage;
};

DockingContext context = new DockingContext(dockableFactory, stageFactory);
DockingPane pane = context.createPane();
context.openDockable(pane, "editor", DockingPosition.CENTER);
context.openDockable(pane, "inspector", DockingPosition.RIGHT);
```

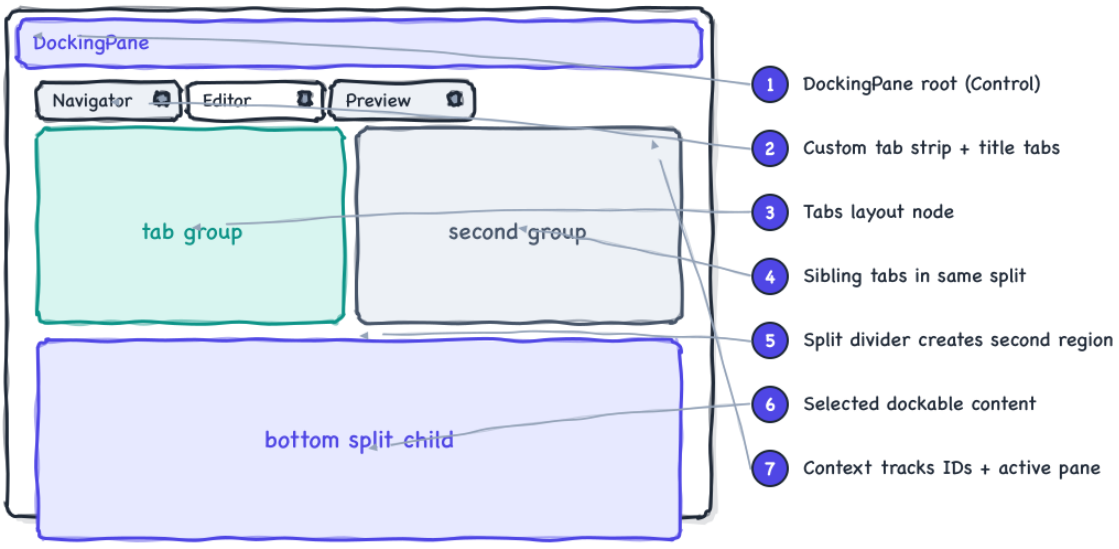
Minimal setup with an ID-based DockableFactory and a shared DockingStageFactory.



Dragging a title reorders tabs, creates splits, or moves a dockable into another pane.

> **Note.** DockingContext methods mutate model state and are expected to run on the JavaFX application thread.

3. Anatomy



Main structural parts of DockingPane.

Part	Type	Description
DockingPane	Control	Root control with style class `docking-pane` and stylesheet `docking-pane.css`.
DockingContext	Model coordinator	Tracks open dockables, pane membership, close policy, floating stages, and snapshot/restore.
Tab group	Layout node	A set of dockable IDs plus one selected ID.
Split node	Layout node	Two child nodes with orientation and divider position.
Floating pane	DockingPane	Created by the context when a dockable is detached to a stage.

The public layout tree is represented by immutable `DockingLayoutNode` variants: `Empty`, `Tabs`, and `Split`.

4. Control API

4.1 DockingContext

Property	Type	Default	Description
createPane()	DockingPane	new pane	Creates a pane attached to the context.
openDockable(pane, id, position)	Optional<Dockable>	n/a	Opens the requested dockable ID or activates the existing one if already open.
floatDockable(id, x, y)	DockingPane	n/a	Moves one open dockable into a new floating pane and positions its stage.
getLayout()	DockingLayout	snapshot	Captures all panes and floating stage bounds.
restoreLayout(layout)	DockingRestoreResult	n/a	Rebuilds the context from a snapshot and reports unresolved IDs.
canCloseDockable policy	Predicate<Dockable>	always true	Veto hook used by close buttons, keyboard close, and floating stage close.

4.2 DockingPane

Property	Type	Default	Description
activeDockable	ReadOnlyObjectProperty<Dockable>	null	Currently active dockable of this pane.
layoutRoot	ReadOnlyObjectProperty<DockingLayoutNode>	Empty	Immutable snapshot of this pane tree, updated on structure and selection changes.
selectDockable(id)	void	n/a	Activates a dockable by ID.
updateDividerPosition(path, position)	void	n/a	Stores divider movement for snapshot fidelity.
moveDockable(...)	boolean	false if invalid	Moves a dockable by ID relative to a target dockable.

5. Tab and split interactions

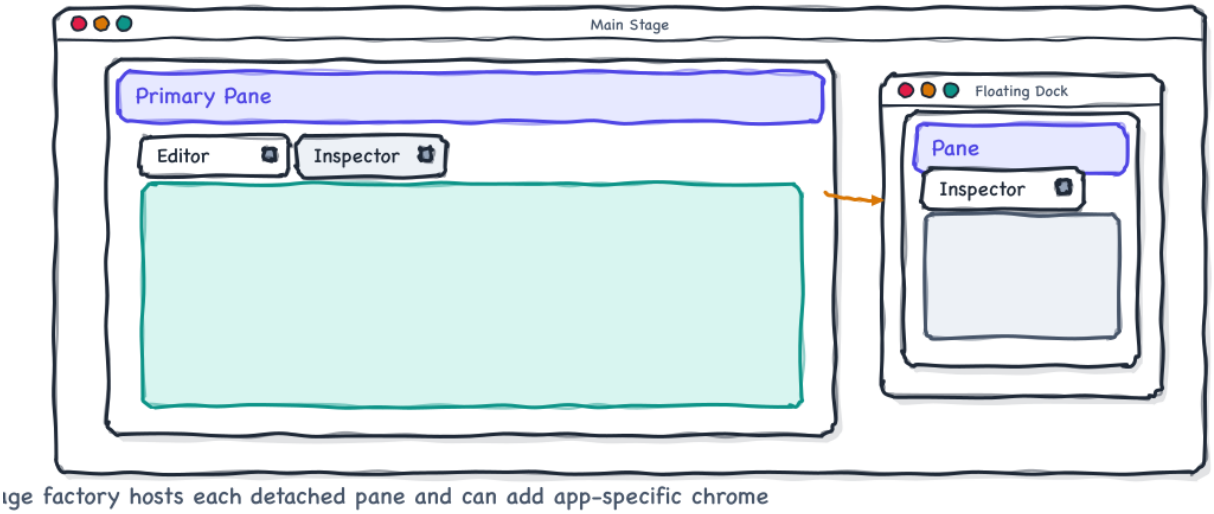
Dragging a title inside the pane resolves a target from cursor position and the currently hovered title. Center drops merge into tabs, edge drops create a split.

Gesture	Result
Drop on center zone	Dockable joins target tab group and becomes selected.
Drop on left/right zone	Horizontal split is created around the target group.
Drop on top/bottom zone	Vertical split is created around the target group.
Drop on another pane (same context)	Cross-pane move; source pane may become empty.

Gesture	Result
ESC during drag	Cancels external floating; no detachment happens.

Keyboard interactions of the title controls: LEFT/RIGHT switches selected tab, ENTER or SPACE selects the focused title, DELETE or shortcut+W requests close.

6. Floating stages

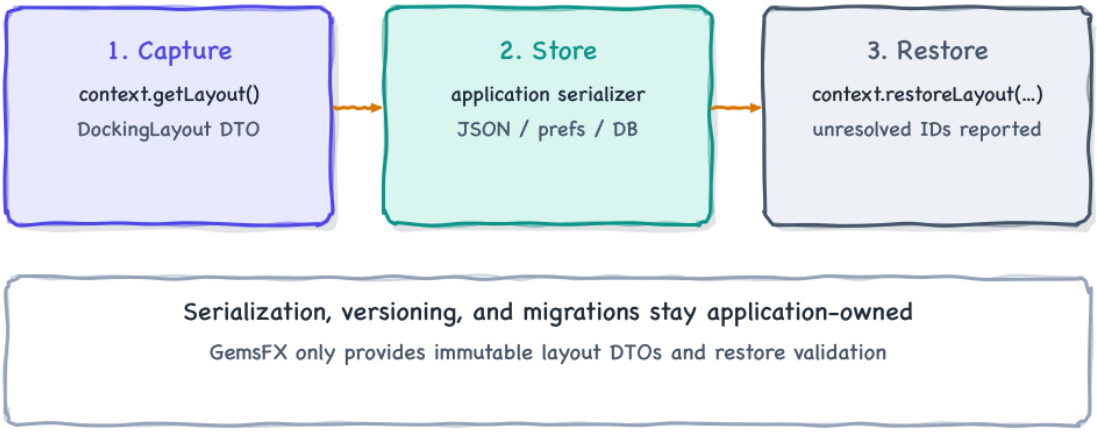


A detached dockable is hosted by a stage produced by the shared stage factory.

When a drag ends outside all context panes, or when the app calls `floatDockable(...)`, the context creates a fresh pane and asks the stage factory to host it. The factory must return a stage that already contains the supplied pane.

- Floating stages are tracked per host stage and close automatically when the host closes.
- If the last dockable leaves a floating pane, its stage is closed automatically.
- Close-veto policy is respected before a floating stage closes.

7. Persistence



Applications own persistence: capture, serialize externally, deserialize, then restore.

DockingContext intentionally keeps serialization/versioning outside the library. Use `getLayout()` to capture a DTO graph, then serialize it in your own format.

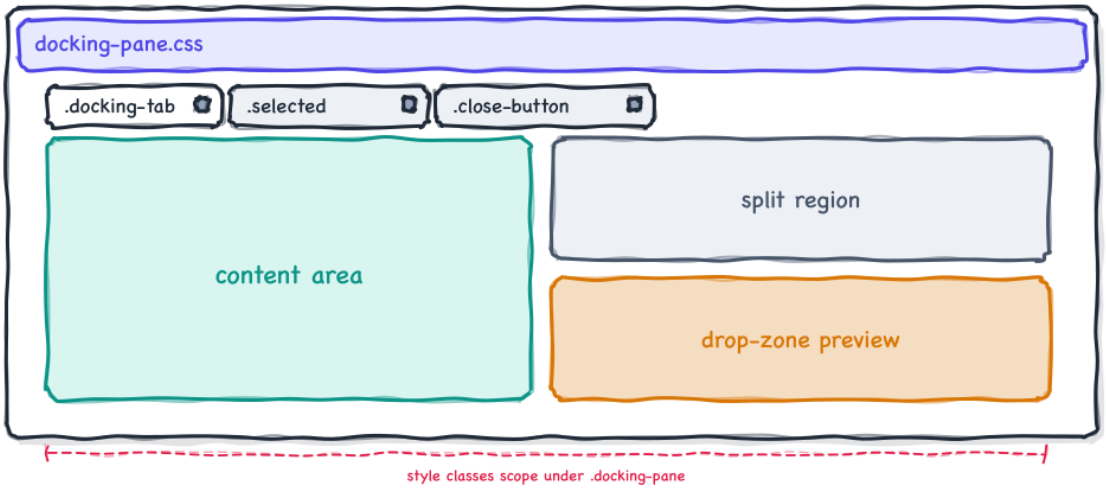
```
DockingLayout layout = context.getLayout();
String json = mySerializer.write(layout); // application-owned

DockingLayout restored = mySerializer.read(json);
DockingRestoreResult result = context.restoreLayout(restored);
if (!result.getUnresolvedDockableIds().isEmpty()) {
    log.warn("Missing dockables: {}", result.getUnresolvedDockableIds());
}
```

Snapshot and restore without introducing a library-side serializer.

> **Note.** Restore validates the layout shape and divider range, and requires exactly one primary pane (the pane layout with null stage bounds).

8. Styling



Main style classes for tabs, split panes, and drop previews.

Selector	Meaning
`.docking-pane`	Control root background and border.
`.docking-tab-strip`	Tab strip container and spacing.
`.docking-tab`	Custom title tab shape and hover/selected visuals.
`.docking-tab-close-button`	Close affordance and icon.
`.docking-split`	SplitPane wrappers created for split layout nodes.
`.docking-drop-leaf-preview`	Outline of target leaf during drag.
`.docking-drop-zone-preview`	Filled drop zone preview.

```
.docking-pane .docking-tab.selected .docking-tab-title {
    -fx-font-weight: bold;
}
.docking-pane .docking-drop-zone-preview {
    -fx-background-color: -fx-accent;
    -fx-opacity: 0.35;
}
```

Excerpt from `docking-pane.css`.

9. Accessibility and localization

DockingPane sets `AccessibleRole.NODE` and a localized role description from resource key `accessible.role-description`.

- Each title tab uses `AccessibleRole.TAB_ITEM` and binds accessible text to the dockable title.
- Close buttons use localized tooltip text (`tooltip.close`).
- Close buttons format accessible text through `accessible.text.close` (for example: *Close Inspector*).

Resource key	Default value
`accessible.role-description`	Docking pane

Resource key	Default value
`accessible.text.close`	Close {0}
`tooltip.close`	Close

10. Basic app recipe

1. Define a stable ID catalog for all dockables your app can create.
2. Implement `DockableFactory` that maps IDs to new Dockable instances.
3. Implement one shared `DockingStageFactory` that wraps supplied panes in app-specific stage chrome.
4. Create one DockingContext, one primary DockingPane, and open the initial dockables.
5. Persist `DockingLayout` snapshots using your own serializer and restore on startup.

```
DockingContext context = new DockingContext(dockableFactory, stageFactory);
DockingPane rootPane = context.createPane();

context.openDockable(rootPane, "editor", DockingPosition.CENTER);
context.openDockable(rootPane, "navigator", DockingPosition.LEFT);
context.openDockable(rootPane, "console", DockingPosition.BOTTOM);

// later
DockingLayout snapshot = context.getLayout();
context.restoreLayout(snapshot);
```

Minimal recipe aligned with the DockingPane demo application.

11. See also

- Demo application: `com.dlsc.gemsfx.demo.DockingPaneApp` (run: `mvn javafx:run -f gemsfx-demo/pom.xml -Dmain.class=com.dlsc.gemsfx.demo.DockingPaneApp`).
- Related APIs: `DockingContext`, `DockingLayout`, `DockingLayoutNode`, `DockingRestoreResult`.
- API documentation: https://dlsc-software-consulting-gmbh.github.io/GemsFX/api/